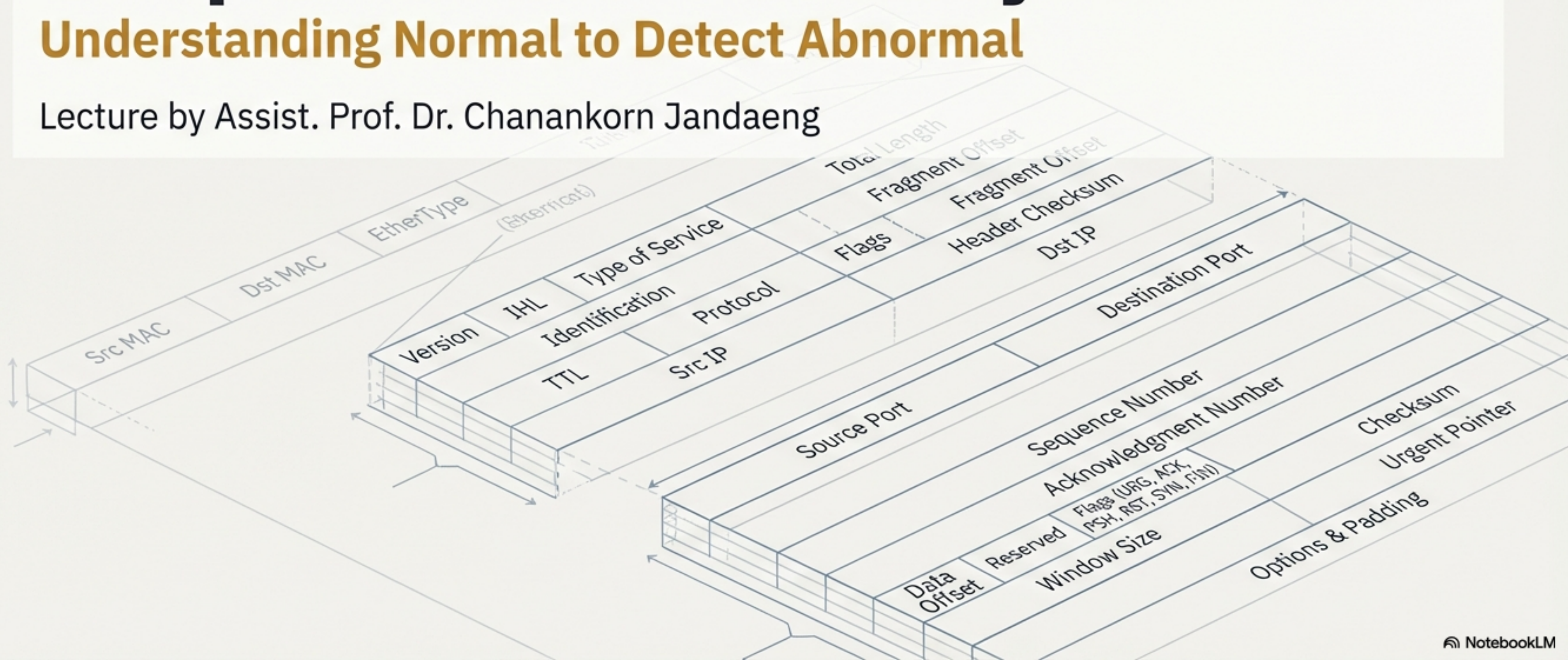


Chapter 3: Traffic Analysis

Understanding Normal to Detect Abnormal

Lecture by Assist. Prof. Dr. Chanankorn Jandaeng



The First Principle of Detection: Know Your Normal

“You can’t detect abnormal if you don’t understand normal.”

Analyzing normal traffic establishes a **baseline**—a clear picture of expected patterns, protocols, and communication channels on your network. This baseline is the essential foundation for all effective intrusion detection. Without it, distinguishing a genuine threat from routine activity is impossible.

Core Goals:

- Reveal expected protocols and services.
- Validate security policies.
- Tune network performance.

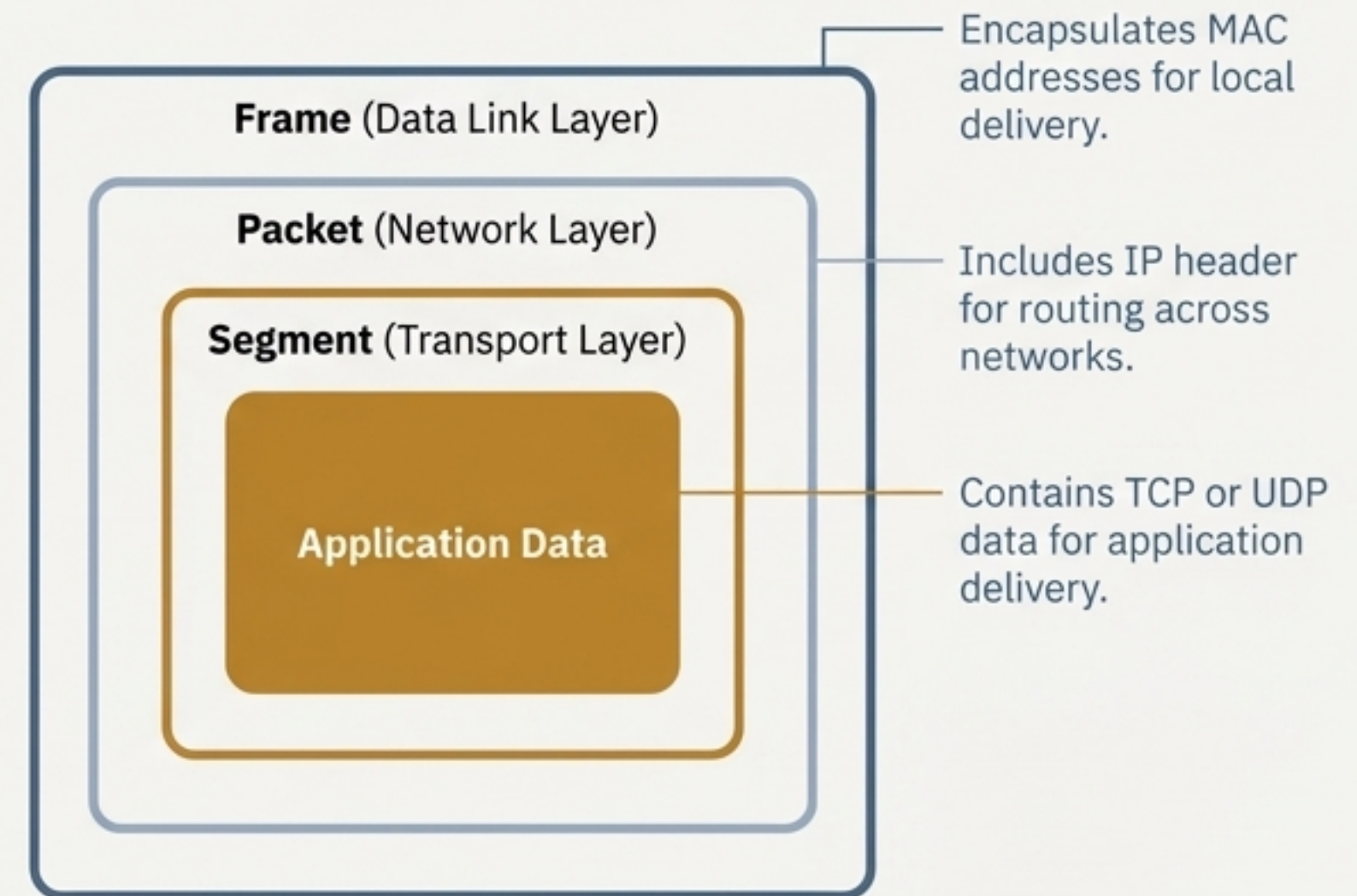
What We See: Deconstructing Network Conversations

Definition

Network Traffic: The flow of data packets across a network between devices.

Key Characteristics: Direction (inbound/outbound), Type (unicast, broadcast, multicast), Volume, and Timing.

The Anatomy of a Packet



The Analyst's Magnifying Glass: An Introduction to Wireshark



Wireshark is an open-source **network protocol analyzer** used to capture and inspect packets in real-time.

Core Capabilities



Live Capture & Offline Analysis:

Analyze traffic as it happens or from saved files.



Deep Packet Inspection: Dissect protocols from Layer 2 (Ethernet) to Layer 7 (Application).



Advanced Filtering & Export:

Isolate specific conversations and export data for reporting.

Common Use Cases: Network Troubleshooting, Performance Monitoring, Security Auditing.

Navigating the Wireshark Interface

The screenshot shows the Wireshark interface with four numbered callouts:

- 1. Menu / Toolbar:** Located at the top of the window, containing icons for file operations, capture, analysis, and view settings.
- 2. Packet List Pane:** A table listing captured packets. The selected packet (No. 5) is highlighted in orange.
- 3. Packet Details Pane:** A hierarchical tree view showing the protocol layers of the selected packet. The 'Transmission Control Protocol' layer is expanded and highlighted in orange.
- 4. Packet Bytes Pane:** A pane showing the raw data of the selected packet in hexadecimal and ASCII format. The data is highlighted in orange.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.001630	197.8.2.1	192.168.255	TCP	68	Sample Fireword 337 (1)
2	0.601526	197.8.2.3	192.168.235	TCP	183	Sample Raqu TCP Conter
3	0.002207	197.8.2.2	192.169.1	TCP	100	Got-received IP Packet
4	0.003298	192.168.255	197.0.2.5	TCP	82	HTTP ERL List Data
5	0.003391	192.168.255	197.0.2.1	TCP	89	TCP Packet, 308, 08f 1
6	0.003533	197.8.8.5	197.0.2.5	TCP	69	Addresser and Carp Yoe
7	0.003583	197.0.2.1	197.0.0.2	TCP	489	Addresser and Contest

Frame 5: 1514 bytes
> Ethernet II, Src: 00:00:10:fa1500, Rht: 8f-adz-de0 (00:00:08:4f:f5:f8cf1)
> Internet Protocol Version 4, Src: 192.168.255.25, 18
v Transmission Control Protocol, Src Port, Atc, Det: Metn [ota], C0, T8]
Source Port: 80 (http)
Destination Port: 50000
Sequence Number: 2519
Sequence Number (Sequence: 18626)

0090 45 00 85 d4 6d 5a 40 00 E5 00 05 d4 6d 5a 40 00 ...E... H..F...
0010 00 00 85 09 6d 39 00 00 00 00 09 00 00 09 00 09 E...mZg
0020 72 01 64 72 87 01 60 55 00 60 96 00 52 66 70 72H.. T...E..
0030 32 00 00 00 60 00 70 60 64 74 90 00 00 00 00 09 ...mZ@.
0040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Key Tip: Understand the pane hierarchy: Start with the Packet List, then dive into the Details. The dissection follows the protocol layers: **Frame** → **Ethernet** → **IP** → **TCP/UDP** → **Application**

The Two Filter Types: What You Capture vs. What You See



Capture Filters

- **Purpose:** Limit what packets are saved to disk. Applied *before* capture starts.
- **Syntax:** Based on **libpcap** rules.
- **Use Case:** Essential for long-term captures on high-traffic networks to save storage and processing power.
- **Example:**

```
host 8.8.8.8  
port 80
```



Display Filters

- **Purpose:** Hide or show packets already present in the capture. Applied *after* capture.
- **Syntax:** **Wireshark's** powerful internal language.
- **Use Case:** The primary tool for interactive analysis, allowing you to dynamically drill down into the data.
- **Example:**

```
ip.addr == 192.168.1.10  
http.request
```


Practical Filtering: Finding the Signal in the Noise

Powerful Display Filters

`ip.addr == 192.168.1.10`

Traffic to or from a specific IP

`dns && udp.port == 53`

DNS queries over UDP

`http.request.method == "POST"`

Find form submissions

`tcp.flags.syn == 1 && tcp.flags.ack == 0`

Find the start of new TCP sessions

`frame.len > 1000`

Isolate large packets

****Pro Tip**:** Combine filters with logical operators:

`ip.src == 10.0.0.1 && tcp.port == 443`

Essential Capture Filters

`host 8.8.8.8`

Traffic involving Google's DNS

`net 192.168.1.0/24`

Traffic within a local subnet

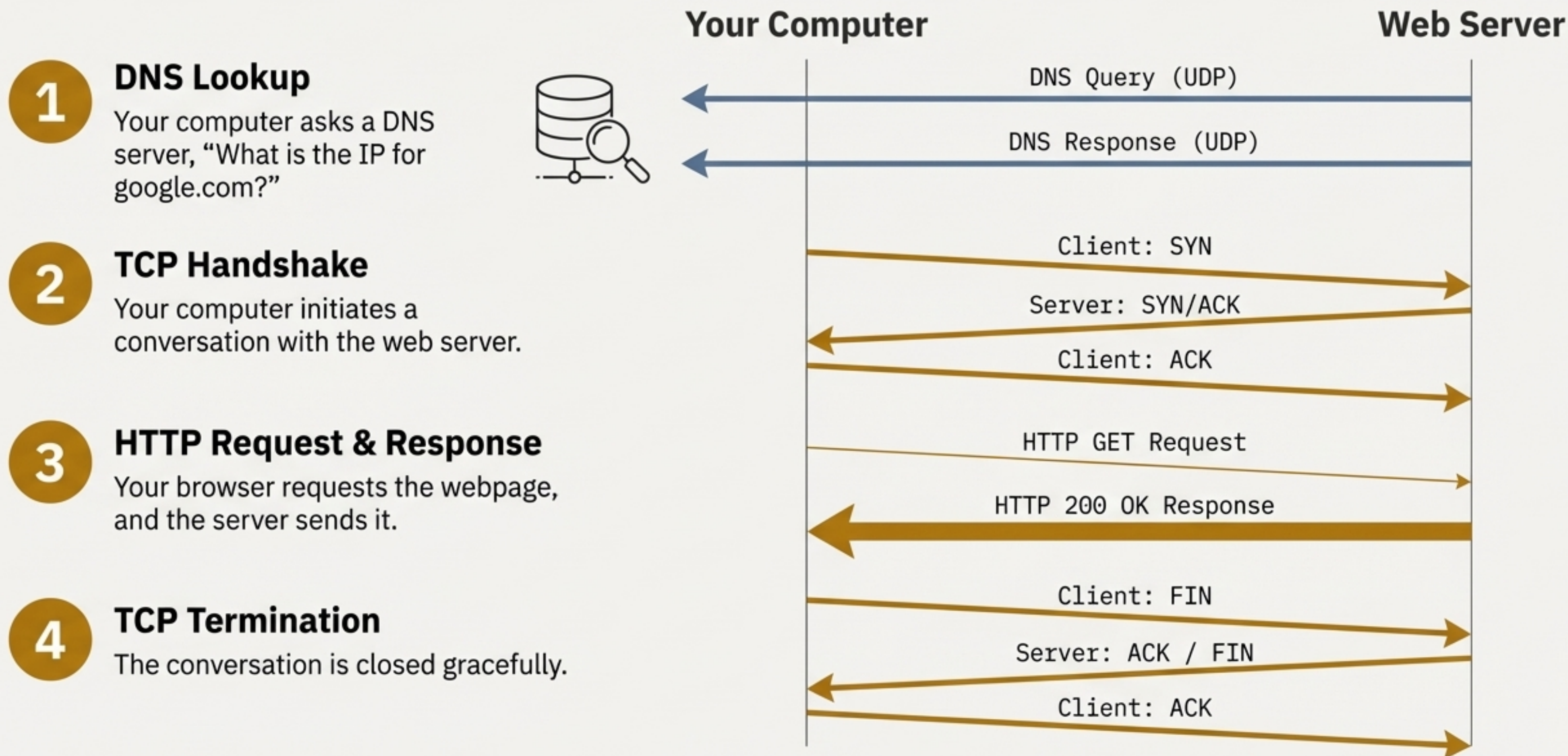
`port 443`

Only HTTPS traffic

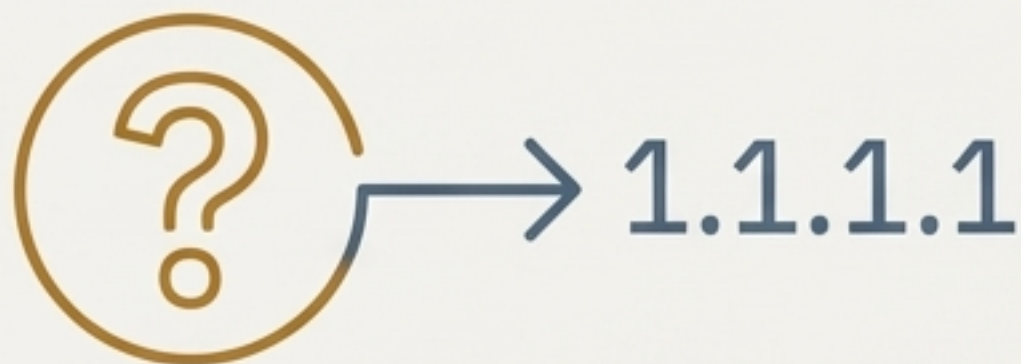
`tcp or udp`

Exclude non-TCP/UDP protocols

Anatomy of a Normal Web Session



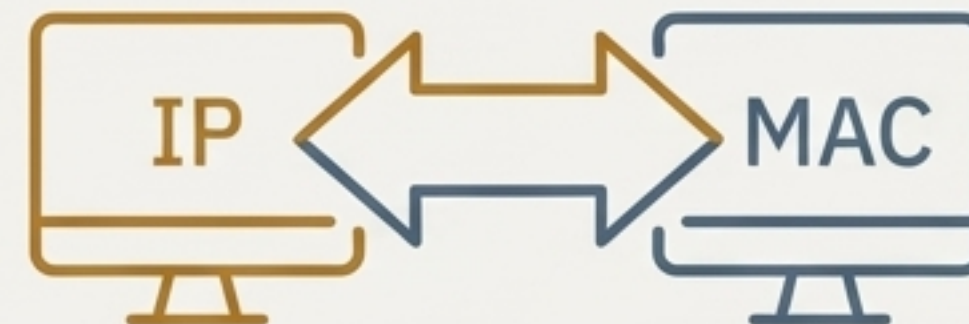
The Rhythms of a Healthy Network: Common Protocol Patterns



DNS (UDP Port 53)

Behavior: Short query/response pairs.

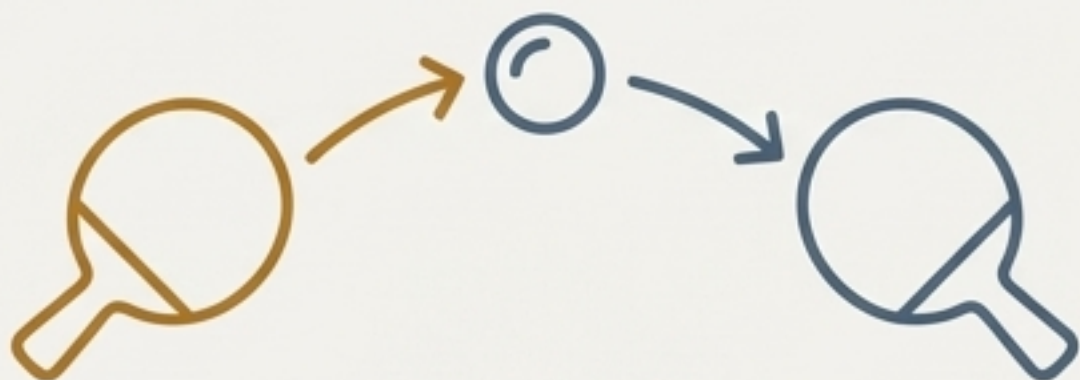
Normal Indicators: Small payloads (<512 bytes), quick responses.



ARP (N/A)

Behavior: Resolves IP addresses to MAC addresses on the local network.

Normal Indicators: Regular, low-volume broadcasts. A sudden spike can indicate scanning.



ICMP (N/A)

Behavior: Used for diagnostics like `ping`.

Normal Indicators: Low frequency, balanced request/reply pairs (Echo/Echo Reply).



TCP

Behavior: Connection-oriented sessions for reliable data transfer.

Normal Indicators: Proper 3-way handshake (SYN, SYN/ACK, ACK) and graceful closure (FIN). Minimal retransmissions.

Beyond the GUI: Automating Analysis with Tshark



Tshark is the command-line engine of Wireshark.

Core Advantages

-  **Lightweight & Fast:** Runs on servers, embedded devices, and headless systems where a GUI is unavailable.
-  **Scripting & Automation:** Integrates perfectly into shell scripts for repeatable, scheduled analysis.
-  **Remote Capture:** The standard for capturing traffic on remote machines over SSH.

Primary Use Case: When you need to move from interactive exploration to automated, batch-oriented data collection and analysis.

Tshark in Action: From Capture to CSV

1. Live Capture & Save

Capture traffic from `eth0` and save it.

```
tshark -i eth0 -w hourly_capture.pcapng -a duration:3600
```

2. Read & Apply Display Filter

Find all DNS queries in a saved file.

```
tshark -r hourly_capture.pcapng -Y "dns.flags.response == 0"
```

3. Extract Specific Fields for Analysis

This is the key automation technique. Create a custom CSV of IP addresses and ports.

```
tshark -r capture.pcap -T fields -e ip.src -e ip.dst -e tcp.dstport  
-E header=y -E separator=, > traffic_log.csv
```

Annotation



-T fields: Specifies custom output format.

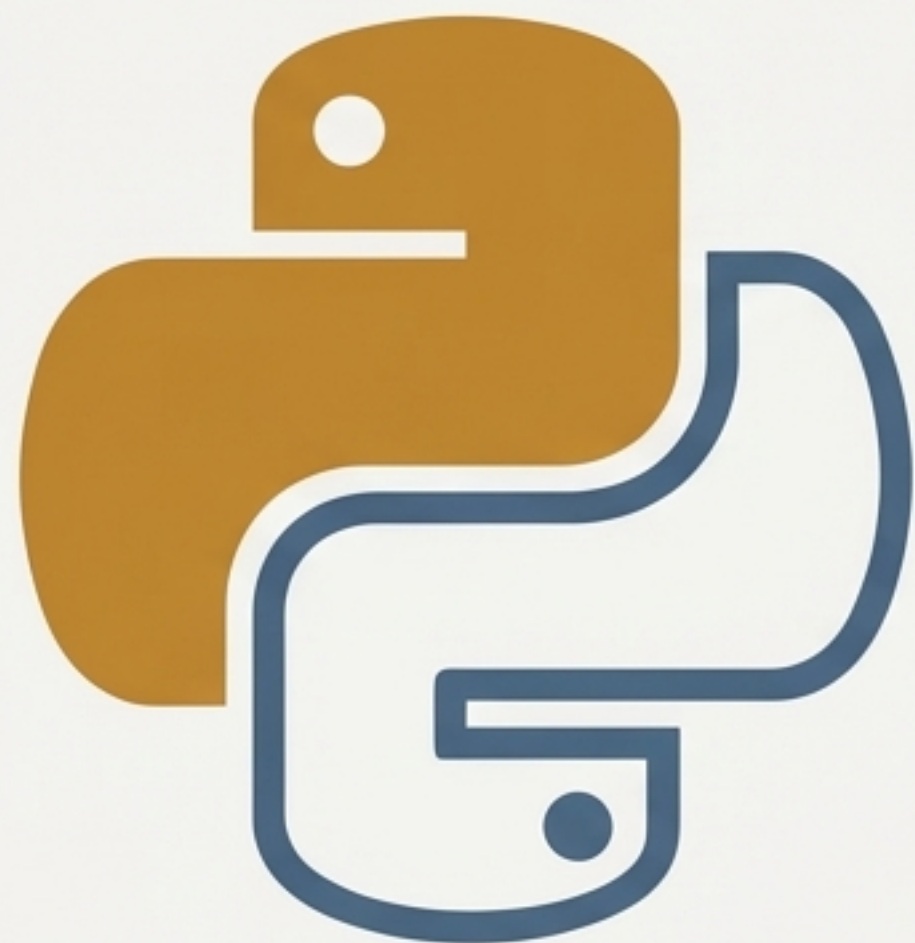


-e: Extracts a specific protocol field.



-E header=y: Adds a CSV header row.

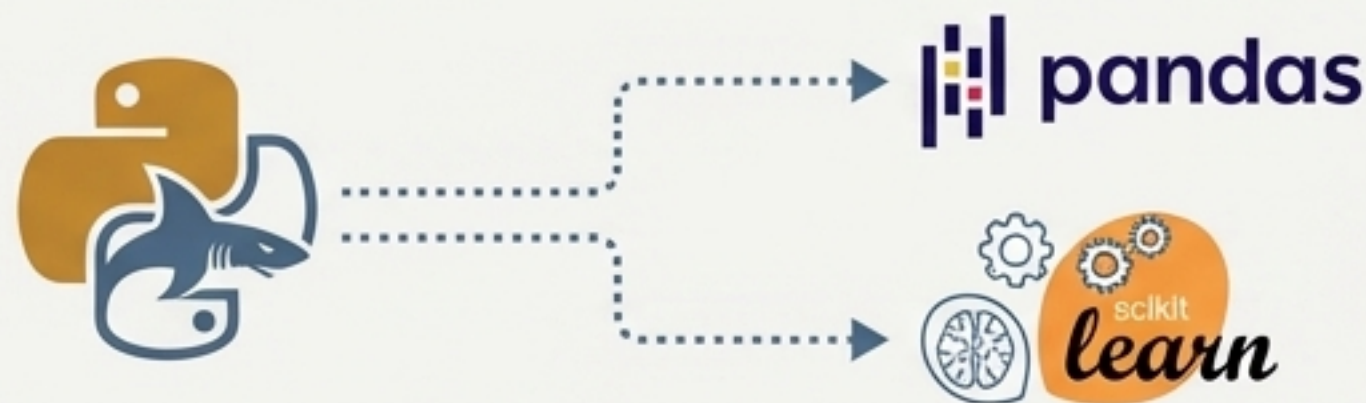
From Automation to Research: Analyzing Packets with PyShark



PyShark is a Python wrapper for Tshark, bringing packet analysis directly into your code.

Purpose

- ⚙️ **Automate** complex analysis workflows that are difficult with shell scripts alone.
- 📄 **Extract** structured data (e.g., features for a model) from PCAP files.
- 🔗 **Integrate** traffic data directly with libraries like Pandas, NumPy, and Scikit-learn.



Core Advantage: Moves beyond simple logging to enable data-driven cybersecurity research and the development of machine learning-based detection systems.

A Glimpse into PyShark: Code-Driven Packet Analysis

1. Reading and Filtering a PCAP File

```
import pyshark

# Filter for only HTTP GET requests in an existing file
cap = pyshark.FileCapture(
    'web_traffic.pcap',
    display_filter='http.request.method == "GET"'
)

for packet in cap:
    print(f"Request to Host: {packet.http.host}")
```

2. Building a Dataset for Analysis

```
import pyshark
import csv

# Live capture of DNS traffic, exporting query names to a CSV
capture = pyshark.LiveCapture(
    interface='eth0',
    display_filter='dns.flags.response == 0'
)

with open('dns_queries.csv', 'w') as f:
    writer = csv.writer(f)
    writer.writerow(['timestamp', 'source_ip', 'query_name'])
    for packet in capture.sniff_continuously(packet_count=50):
        writer.writerow([
            packet.sniff_time,
            packet.ip.src,
            packet.dns.qry_name
        ])
    ])
```


The Analyst's Toolkit: Choosing the Right Tool for the Job



Wireshark



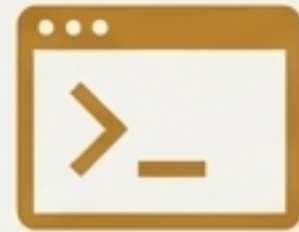
Interface: Graphical (GUI)



Strength: Intuitive, interactive exploration and deep visual dissection.



Best For: Local, hands-on analysis and troubleshooting.



Tshark



Interface: Command-Line (CLI)



Strength: Lightweight, scriptable, and fast.



Best For: Remote capture, automated batch processing, and server-side analysis.



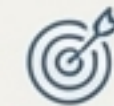
PyShark



Interface: Python Code



Strength: Full programmatic control, integration with data science libraries.



Best For: Building custom analysis tools, creating datasets for ML, and reproducible research.

From Raw Data to Actionable Insight



The Journey Recap

-  **Explore** with **Wireshark**: Visually dissect traffic to understand conversations.
-  **Automate** with **Tshark**: Systematically capture and filter data at scale.
-  **Integrate** with **PyShark**: Build custom tools and datasets for advanced analysis.

The Foundation is Set

You now have the toolkit to define ‘normal’ for any network. This baseline is your ground truth.

Next Chapter: We will now use this foundation to hunt for the abnormal: the anomalies, threats, and attacks hiding in the traffic.